

PROYECTO DE MONITOREO VEHICULAR

Despliegue de la plataforma web VehicleSense en AWS EC2

Informe técnico final sobre arquitectura cloud, operación y validación

Patricio Vasquez

31 de agosto de 2026

Resumen

Este informe documenta el despliegue productivo de VehicleSense, una plataforma de monitoreo vehicular que integra un dispositivo ESP32, sensores, transporte MQTT, un backend FastAPI, PostgreSQL y una interfaz React. La aplicación fue publicada en una instancia AWS EC2 Ubuntu 24.04 ARM64 mediante Docker Compose. Nginx actúa como punto único de entrada y entrega el frontend, la API REST y el canal WebSocket; HiveMQ Cloud permanece como broker MQTT externo.

Se describen la arquitectura, la configuración de red, el uso de Elastic IP, Security Groups, DNS de Namecheap, certificados Let's Encrypt, migraciones Alembic, backups y operación diaria. La elección de AWS EC2 se fundamenta en el control sobre el sistema Linux y la red, la compatibilidad con ARM64, la integración de sus servicios y el valor formativo de administrar directamente el servidor. La validación confirmó el acceso HTTPS, los endpoints de salud, la conexión del backend con PostgreSQL y HiveMQ, y la independencia de la Mac local.

El despliegue queda operativo como una topología de una sola instancia. Por ello, la disponibilidad observada no equivale a alta disponibilidad y la precisión de los sensores, la ejecución de comandos del firmware y la validación física del dispositivo permanecen fuera del alcance de este informe.

Palabras clave: AWS EC2, Docker Compose, Nginx, FastAPI, MQTT, HiveMQ, PostgreSQL, React, HTTPS, monitoreo vehicular.

Índice general

Resumen	I
1. Introducción	1
1.1. Planteamiento	1
1.2. Objetivos	1
1.3. Alcance y criterio de evidencia	2
2. Contexto técnico del proyecto	3
2.1. Flujo de información	3
2.2. Componentes desplegados	3
3. Arquitectura en AWS	5
3.1. Recursos de la instancia	5
3.2. Red y exposición de puertos	5
3.3. DNS y dominio	6
3.4. Persistencia y disponibilidad	6
4. Proceso de despliegue y manejo de AWS	7
4.1. Preparación de Ubuntu	7
4.2. Configuración de variables	7
4.3. Primer arranque y migraciones	8
4.4. HTTPS con Let's Encrypt	8
4.5. Renovación y operación	8
5. Herramientas y decisiones tecnológicas	9
5.1. Por qué AWS EC2	9
5.2. Comparación con alternativas	9
5.3. Herramientas utilizadas	10

6. Seguridad, mantenimiento y costos	12
6.1. Controles aplicados	12
6.2. Backups y recuperación	12
6.3. Memoria, costos y continuidad	13
7. Validación y resultados	14
7.1. Validaciones ejecutadas	14
7.2. Incidentes y lecciones aprendidas	15
7.3. Resultado funcional	15
8. Procedimiento operativo	16
8.1. Comprobación diaria	16
8.2. Actualización controlada	16
8.3. Registro de dispositivos	16
9. Conclusiones	17
A. Checklist de primer despliegue	18

Capítulo 1

Introducción

1.1. Planteamiento

VehicleSense nació como un prototipo académico para observar variables de un vehículo y presentarlas en una interfaz web. El sistema combina mediciones de temperatura, humedad, posición, aceleración, iluminación y características acústicas. Para que la información pueda consultarse sin mantener un computador personal encendido, fue necesario trasladar la capa cloud a una infraestructura conectada permanentemente a Internet.

El reto no consistió únicamente en copiar el frontend a un servidor. La solución debía recibir mensajes MQTT sobre TLS, validar los contratos de telemetría, persistir el historial, autenticar usuarios, servir una API y mantener una conexión WebSocket para actualizaciones en vivo. Además, el acceso público debía utilizar HTTPS y no debía exponer PostgreSQL, FastAPI ni el frontend interno directamente.

1.2. Objetivos

1. Publicar la aplicación web en el dominio <https://vehiclemonitorsense.me> con HTTPS.
2. Ejecutar frontend, backend y PostgreSQL en una instancia EC2 reproducible mediante Docker Compose.
3. Conectar FastAPI con HiveMQ Cloud por MQTT/TLS en el puerto 8883.
4. Proteger la superficie pública con Security Groups, Nginx, TLS, roles y separación de credenciales.
5. Validar el funcionamiento mediante health checks, migraciones, autenticación, pruebas de red y procedimientos de backup.

1.3. Alcance y criterio de evidencia

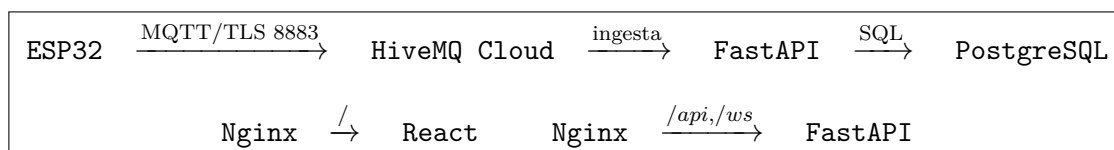
Este documento cubre el despliegue de la plataforma web y su operación inicial. Cuando se indica que una característica está implementada, existe configuración o código versionado. Cuando se indica que está validada en producción, existe una comprobación fechada en la infraestructura desplegada. La validación de precisión de sensores, el bring-up eléctrico, la calibración acústica y la ejecución semántica de comandos no se presentan como resultados del despliegue cloud.

Capítulo 2

Contexto técnico del proyecto

2.1. Flujo de información

El flujo completo separa el dispositivo de la aplicación web. El ESP32 adquiere y valida muestras; la telemetría se serializa según los esquemas del proyecto y se publica mediante MQTT/TLS. FastAPI recibe los mensajes desde HiveMQ, verifica identidades, tópicos, tamaño, esquema y valores finitos, y luego persiste los datos en PostgreSQL. React consume REST y WebSocket a través de Nginx.



Los tópicos canónicos utilizados por el backend son:

```
vehiclesense/v1/vehicles/{vehicle_id}/devices/{device_id}/telemetry
vehiclesense/v1/vehicles/{vehicle_id}/devices/{device_id}/status
vehiclesense/v1/vehicles/{vehicle_id}/devices/{device_id}/events
vehiclesense/v1/vehicles/{vehicle_id}/devices/{device_id}/acoustic
vehiclesense/v1/vehicles/{vehicle_id}/devices/{device_id}/commands
vehiclesense/v1/vehicles/{vehicle_id}/devices/{device_id}/command-acks
```

2.2. Componentes desplegados

Componente	Función en el proyecto	Ubicación o tecnología
ESP32	Adquisición, validación, almacenamiento local y publicación de datos	Hardware del prototipo
HiveMQ Cloud	Autenticación, ACL, TLS y transporte MQTT	Servicio externo
FastAPI	Ingesta, autenticación, REST, WebSocket, alertas y comandos	Contenedor Docker
PostgreSQL	Persistencia de usuarios, dispositivos, telemetría, alertas y viajes	Contenedor Docker
React/Vite	Dashboard, mapas, gráficas, alertas y reportes	Contenedor estático Nginx
Nginx	Entrada HTTP/HTTPS, proxy REST/WS y redirecciones	Contenedor público

Cuadro 2.1: Componentes de la solución desplegada.

Capítulo 3

Arquitectura en AWS

3.1. Recursos de la instancia

La plataforma se desplegó en una instancia dedicada de producción, con Ubuntu Server 24.04 LTS sobre arquitectura ARM64. Se seleccionó una instancia `t4g.small`, con 2 vCPU y 2 GiB de RAM, y un volumen raíz gp3 de 30 GiB. El diseño no fija una arquitectura en Docker, por lo que también es compatible con EC2 AMD64.

La instancia tiene una Elastic IP asociada. Esto permite que los registros DNS continúen apuntando a una dirección estable aunque la instancia se detenga o se reemplace. La Mac del desarrollador no participa en la ejecución: únicamente se utiliza para administrar el servidor por SSH cuando es necesario.

3.2. Red y exposición de puertos

Se creó un Security Group dedicado. Las reglas de entrada se limitaron a lo estrictamente necesario:

Servicio	Puerto	Origen o propósito
SSH	22/TCP	Solo la IP pública actual del administrador, en formato /32
HTTP	80/TCP	Público para redirección y desafío ACME
HTTPS	443/TCP	Público para la aplicación web
PostgreSQL	5432/TCP	No publicado; red Docker interna
FastAPI	8000/TCP	No publicado; solo Nginx y la red Docker
Frontend	8080/TCP	No publicado; solo Nginx
MQTT	8883/TCP	No es entrada EC2; conexión saliente a HiveMQ

Cuadro 3.1: Puertos y frontera de red de EC2.

La red Docker `database` se define como interna. FastAPI participa tanto en la red privada de base de datos como en la red de borde, mientras que PostgreSQL solo participa en la

red privada. Nginx es el único servicio que publica puertos del host.

3.3. DNS y dominio

En Namecheap se reemplazaron los registros de parking por registros A para el dominio raíz y para `www`. Ambos registros apuntan a la Elastic IP de la instancia. La propagación se comprobó con `dig` antes de solicitar el certificado. El comportamiento final es:

- `http://vehiclemonitorsense.me` redirige a HTTPS.
- `http://www.vehiclemonitorsense.me` redirige a HTTPS.
- `https://www.vehiclemonitorsense.me` redirige al dominio principal.

3.4. Persistencia y disponibilidad

PostgreSQL se ejecuta con un volumen Docker persistente denominado `postgres_data`. Alembic aplica las migraciones antes de iniciar FastAPI; si la migración falla, el backend no inicia. Los backups se generan con `pg_dump` en formato custom, se acompañan de un checksum SHA-256 y deben copiarse a un almacenamiento cifrado externo.

La topología es de una sola instancia. Esto simplifica el proyecto académico y reduce la cantidad de componentes, pero introduce un punto único de fallo. No se afirma alta disponibilidad, balanceo ni recuperación automática entre zonas.

Capítulo 4

Proceso de despliegue y manejo de AWS

4.1. Preparación de Ubuntu

Después de lanzar la instancia y asociar la Elastic IP, se accedió por SSH con la llave privada protegida mediante permisos 600. En el servidor se creó `/opt/vehiclesense`, se clonó el repositorio y se ejecutó el script versionado `bootstrap_ubuntu.sh`. El script instala Docker Engine, Buildx, Compose, Git, OpenSSL, jq y los certificados base, y agrega el usuario `ubuntu` al grupo `docker`.

La sesión SSH se cerró y se abrió nuevamente para que el grupo se aplicara. Las versiones observadas en producción fueron Docker Engine 29.7.2 y Docker Compose v5.5.0 sobre `linux/arm64`.

4.2. Configuración de variables

El archivo `/opt/vehiclesense/deploy/.env` no se versiona y se protegió con `chmod 600`. Se configuraron:

- dominio principal, dominio alternativo y correo de Let's Encrypt;
- credenciales y nombre de la base de datos;
- secreto JWT y usuario administrador inicial;
- host, puerto, usuario, contraseña y client ID de HiveMQ;
- orígenes CORS HTTPS y umbrales de estado de dispositivos.

Las credenciales del backend son diferentes de las credenciales compiladas en el ESP32. No se copiaron secretos, certificados ni dumps a Git, capturas o informes. El auto-registro de dispositivos se mantuvo desactivado:

```
NGINX_TEMPLATE=https.conf.template
VEHICLESENSE_AUTO_REGISTER_DEVICES=false
```

4.3. Primer arranque y migraciones

El script `deploy.sh` valida los placeholders, comprueba el formato de la contraseña de PostgreSQL, fuerza MQTT/TLS en el puerto 8883, valida Compose, construye las imágenes y converge los servicios con `-wait`. En el primer arranque se utilizó temporalmente `http.conf.template`; esto permite que Nginx sirva el desafío ACME antes de disponer del certificado.

El entrypoint de FastAPI ejecutó:

```
alembic upgrade head
```

La revisión aplicada fue `cd5ddc949e1d`, que quedó en `head`. El administrador bootstrap se creó correctamente y los cuatro servicios `postgres`, `backend`, `frontend` y `nginx` quedaron **healthy**.

4.4. HTTPS con Let's Encrypt

Con DNS propagado y HTTP disponible, Certbot se ejecutó como servicio temporal de Compose en modo `webroot`. Se emitió un certificado para el dominio principal y `www`. Luego se cambió `NGINX_TEMPLATE` a `https.conf.template`, se recreó Nginx y se comprobó la redirección de `www`, el proxy de la API y el canal WebSocket.

Durante el procedimiento se detectó un incidente de configuración: una sesión SSH con variables exportadas mediante `source .env` conservaba el valor anterior de `NGINX_TEMPLATE` (`http.conf.template`), aunque el archivo ya había sido editado a HTTPS. Compose prioriza las variables exportadas sobre el archivo indicado con `-env-file`. La solución fue cerrar la sesión, abrir una sesión limpia y recrear Nginx usando explícitamente `-env-file .env`. Desde entonces, las operaciones normales utilizan los scripts versionados y no dejan secretos exportados permanentemente en la shell.

4.5. Renovación y operación

La renovación se programó con una entrada cron diaria a las 03:17. El script de renovación ejecuta `certbot renew` y recarga Nginx. Las comprobaciones periódicas incluyen:

```
docker compose --env-file deploy/.env -f deploy/compose.production.yml ps
docker compose --env-file deploy/.env -f deploy/compose.production.yml logs
--tail=100
curl --fail https://vehiclemonitorsense.me/health/live
curl --fail https://vehiclemonitorsense.me/health/ready
df -h
free -h
swapon --show
```

Las actualizaciones se realizan con `update.sh`. Este exige un árbol Git limpio, acepta únicamente fast-forward, genera un backup y guarda la revisión anterior antes de construir la nueva versión.

Capítulo 5

Herramientas y decisiones tecnológicas

5.1. Por qué AWS EC2

AWS EC2 fue elegido por cuatro razones principales:

1. **Control operativo.** Permite administrar Ubuntu, Docker, Nginx, cron, almacenamiento y red para demostrar un despliegue reproducible.
2. **Servicios integrados.** Elastic IP, EBS, Security Groups y VPC resuelven direccionamiento, almacenamiento y control de acceso sin introducir un proveedor adicional para la máquina.
3. **Compatibilidad y flexibilidad.** La familia Graviton ARM64 ofrece una instancia pequeña para comenzar, mientras que el mismo Compose puede ejecutarse en AMD64 o escalar verticalmente si aumentan los recursos.
4. **Continuidad del proyecto.** La cuenta AWS, el dominio y la experiencia previa con Linux/SSH permitieron administrar el sistema con herramientas conocidas y conservar la posibilidad de ampliar la infraestructura más adelante.

La decisión implica trabajo adicional: el sistema operativo, los parches, los backups, los certificados, la monitorización y la seguridad no quedan delegados a una plataforma PaaS. Ese costo operativo se aceptó porque el objetivo del proyecto incluye comprender el ciclo completo de despliegue.

5.2. Comparación con alternativas

La comparación no afirma que AWS sea universalmente superior. Para una aplicación pequeña cuyo único objetivo fuera servir contenido estático, una plataforma PaaS podría reducir el trabajo. En VehicleSense, la combinación de MQTT, persistencia, WebSocket y control de red justificó la elección de una VM administrable.

Alternativa	Ventaja principal	Motivo para no usarla como plataforma final
AWS EC2	Control completo de VM, red, almacenamiento y seguridad; arquitectura portable con Docker	Requiere administrar el servidor y sus costos
Render/Railway	Despliegue sencillo y menor administración del sistema	Menor control de red, almacenamiento y proceso MQTT persistente; costos y límites dependen del plan
Vercel/Netlify	Excelente entrega de frontend estático	No resuelven por sí solos FastAPI, PostgreSQL, MQTT y WebSocket persistente
VPS administrada	Puede ser económica y simple para una VM	Menor integración nativa con la cuenta AWS y menor continuidad con la infraestructura elegida
OCI Compute	Alternativa portable para Docker y máquinas ARM	Se decidió mantener AWS como proveedor de referencia del proyecto

Cuadro 5.1: Comparación cualitativa de plataformas para este proyecto.

5.3. Herramientas utilizadas

Herramienta	Uso realizado
EC2 Console	Lanzamiento de la instancia, selección ARM64, llave y Security Group
Elastic IP	Dirección pública estable para DNS
Namecheap	Registros A para el dominio raíz y www
SSH/OpenSSH	Administración remota segura de Ubuntu
Docker Engine/Compose	Construcción y ejecución reproducible de los servicios
Nginx	Proxy inverso, HTTPS, WebSocket, headers y redirecciones
Certbot/Let's Encrypt	Emisión y renovación automática de certificados TLS
HiveMQ Cloud	Broker MQTT externo con TLS y ACL
FastAPI/PostgreSQL/Alembic	API, ingesta, persistencia y migraciones
React/Vite	Compilación y entrega del dashboard
Git y scripts Bash	Versionado, despliegue, actualización y backup
curl/dig/jq	Diagnóstico de DNS, HTTP, JSON y health checks

Cuadro 5.2: Herramientas principales del despliegue.

Capítulo 6

Seguridad, mantenimiento y costos

6.1. Controles aplicados

La seguridad se planteó por capas:

- el navegador nunca recibe credenciales MQTT ni PostgreSQL;
- el broker utiliza una credencial separada para el backend y TLS en 8883;
- el Security Group no expone PostgreSQL, FastAPI ni el frontend interno;
- Nginx aplica HTTPS, HSTS, CSP, `nosniff`, bloqueo de frames y límite de solicitudes para el login;
- FastAPI valida roles, payloads, tópicos, identidades y duplicados;
- los secretos permanecen en `deploy/.env`, con permisos restrictivos, y no se incluyen en la imagen del frontend;
- el acceso SSH se restringe a la IP del administrador y la llave privada no se almacena en el repositorio.

Estos controles reducen la superficie de ataque, pero no convierten al prototipo en un sistema de seguridad vehicular certificado. Persisten riesgos como JWT sin revocación persistente, ausencia de MFA, falta de ACL por vehículo en la API, OTA local sin firma y una sola instancia.

6.2. Backups y recuperación

El backup lógico se crea antes de una actualización y también puede ejecutarse de forma manual. El archivo dump se acompaña de un checksum y debe copiarse fuera de EC2 a almacenamiento cifrado. La restauración es una operación explícita y destructiva que requiere `CONFIRM_RESTORE=vehiclesense`; primero se detiene FastAPI y luego se reinicia después de `pg_restore`.

No se borra el volumen `postgres_data` para resolver fallos de migración. Antes de cambiar una revisión se revisa la compatibilidad entre el código y las migraciones.

6.3. Memoria, costos y continuidad

La instancia `t4g.small` tiene 2 GiB de RAM. Se recomienda comprobar swap, memoria y disco antes de builds; si se presenta OOM se puede aumentar verticalmente la instancia sin modificar Compose. La Elastic IP y el volumen EBS generan costos según la cuenta y la región, por lo que deben revisarse las alertas de presupuesto y evitarse recursos no utilizados.

La continuidad 24/7 depende de que EC2 permanezca ejecutándose, el dominio conserve los registros DNS, el certificado se renueve y HiveMQ esté disponible. Apagar la Mac, cerrar SSH o cambiar de red doméstica no detiene la plataforma web.

Capítulo 7

Validación y resultados

7.1. Validaciones ejecutadas

La validación se realizó en dos niveles. Primero se comprobó localmente que las imágenes y la configuración fueran reproducibles. Después se verificaron los endpoints públicos y el estado reportado de EC2.

Prueba	Resultado	Alcance
Build Docker	Backend y frontend contruidos para ARM64	Automatizado/local
Compose	Configuración válida; solo Nginx publica 80/443	Automatizado/local
Migraciones	cd5ddc949e1d (head)	PostgreSQL de producción
Servicios	PostgreSQL, backend, frontend y Nginx healthy	Producción reportada
HTTPS/DNS	Dominio principal 200; www redirige; HTTP fuerza HTTPS	Público, 31/08/2026
Health	/health/live=200 y /health/ready=200	Público, 31/08/2026
MQTT	Backend conectado por TLS/8883	Producción reportada y health ready
Frontend/API	Página y login disponibles detrás de Nginx	Local y público
Backup/restore	Dump, checksum y restauración ejecutados localmente	Automatizado/local

Cuadro 7.1: Matriz resumida de validación del despliegue.

La respuesta final de `/health/ready` indicó que la base de datos estaba disponible y que el cliente MQTT estaba conectado. Esto valida la cadena cloud hasta sus dependencias inmediatas, pero no reemplaza una sesión prolongada con el ESP32 ni una medición de precisión de sensores.

7.2. Incidentes y lecciones aprendidas

El principal incidente fue el uso de variables exportadas de una sesión SSH antigua. La shell mantuvo la plantilla HTTP y provocó que Nginx no escuchara en 443 aunque el archivo `.env` ya indicara HTTPS. La solución fue reiniciar la sesión y usar `docker compose -env-file`. La lección es no mezclar variables persistentes de shell con configuración versionada y validar siempre la configuración efectiva de Nginx con `nginx -T`.

También se comprobó que el problema inicial de SSH provenía de la red local que bloqueaba o interfería con TCP/22. El cambio temporal al hotspot permitió acceder a EC2 y ajustar la regla del Security Group a la nueva IP. La web no depende del hotspot; solo la administración SSH puede depender de esa red.

7.3. Resultado funcional

El resultado final es una página web accesible desde Internet, independiente de la Mac, con autenticación, frontend React, API FastAPI, WebSocket, PostgreSQL y conexión MQTT/TLS. Nginx termina HTTPS y mantiene los servicios internos aislados. El despliegue es reproducible desde el repositorio mediante `deploy.sh` y las actualizaciones se realizan con backup previo mediante `update.sh`.

Capítulo 8

Procedimiento operativo

8.1. Comprobación diaria

La secuencia mínima para revisar la plataforma es:

1. entrar por SSH con la llave y la IP permitida;
2. verificar `docker compose ps`;
3. revisar logs recientes de backend y Nginx;
4. consultar `/health/live` y `/health/ready`;
5. revisar disco, memoria, swap y el cron de renovación;
6. confirmar que no existan puertos internos publicados.

8.2. Actualización controlada

Las actualizaciones no se hacen con un `git pull` aislado. Se ejecuta:

```
cd /opt/vehiclesense
./deploy/scripts/update.sh main
```

El script exige un árbol limpio, toma un backup, registra la revisión previa, acepta fast-forward y vuelve a construir el stack. Si el build falla, no ejecuta rollback de la base de datos automáticamente: la decisión se toma revisando migraciones y el backup.

8.3. Registro de dispositivos

Antes de aceptar telemetría real se registran en el backend los identificadores exactos de vehículo y dispositivo. Deben coincidir con el firmware, el tópico MQTT y las ACL de HiveMQ. Mantener `VEHICLESENSE_AUTO_REGISTER_DEVICES=false` evita que un dispositivo desconocido sea aceptado automáticamente.

Capítulo 9

Conclusiones

El despliegue de VehicleSense en AWS EC2 permitió convertir una aplicación local en una plataforma web accesible permanentemente. La solución integra una VM ARM64, Docker Compose, Nginx, FastAPI, PostgreSQL, HiveMQ Cloud y React sin exponer los servicios internos. La configuración de DNS, Elastic IP, Security Group y Let's Encrypt completó la publicación del dominio con HTTPS.

AWS EC2 fue una decisión coherente con el objetivo técnico y académico: ofreció control directo de Linux y de la red, compatibilidad con Docker, herramientas de operación conocidas y una ruta de crecimiento vertical. La contrapartida es que el mantenimiento queda bajo responsabilidad del proyecto, especialmente backups, parches, costos, monitorización y recuperación.

El sistema queda operativo para la siguiente fase: registrar los IDs reales, conectar el ESP32, comprobar ingestión y WebSocket con telemetría física, y documentar una sesión prolongada. Estas tareas amplían la validación del sistema, pero no requieren volver a aprovisionar EC2, DNS, TLS ni HiveMQ.

Apéndice A

Checklist de primer despliegue

#	Verificación
1	Instancia Ubuntu 24.04 ARM64/AMD64 creada y Elastic IP asociada.
2	Security Group con SSH restringido y HTTP/HTTPS públicos; sin 5432, 8000, 8080 ni 8883 de entrada.
3	DNS A de @ y www resuelve a la Elastic IP.
4	Docker Engine y Compose instalados; usuario reconectado al grupo Docker.
5	deploy/.env creado con permisos 600 y sin placeholders.
6	Credenciales de PostgreSQL, JWT, administrador y HiveMQ separadas.
7	NGINX_TEMPLATE=http.conf.template usado solo para el primer desafío ACME.
8	deploy.sh ejecutado; cuatro servicios healthy y Alembic en head.
9	Certbot emitió certificado para dominio raíz y www.
10	NGINX_TEMPLATE=https.conf.template; HTTP redirige a HTTPS.
11	Renovación cron configurada una sola vez y verificada con crontab -l.
12	Bootstrap admin vaciado después de confirmar el acceso.
13	Backup con checksum copiado a almacenamiento externo cifrado.
14	IDs de vehículo/dispositivo registrados y telemetría física validada.

Referencias

- [1] Amazon Web Services, “Security groups for your EC2 instances”, <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html>.
- [2] Amazon Web Services, “Elastic IP addresses”, <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/elastic-ip-addresses-eip.html>.
- [3] Docker, “Docker Compose documentation”, <https://docs.docker.com/compose/>.
- [4] Certbot, “Using Certbot with webroot”, <https://eff-certbot.readthedocs.io/en/stable/using.html>.
- [5] HiveMQ, “HiveMQ Cloud documentation”, <https://www.hivemq.com/docs/hivemq-cloud/>.
- [6] VehicleSense, “Documentación interna del proyecto: arquitectura, seguridad, despliegue y pruebas”, repositorio del proyecto, 2026.